

Programming Exercise 04: Building Stable Columns

Hash Table Application

Data Structure and Database
Shiraz University

December 9, 2025

1 The Problem

You're working as a structural engineer and need to build vertical support columns using rectangular blocks. Each block is like a box with three dimensions (length, width, height), and you can rotate it any way you want. Your goal is to build the most stable column possible.

2 What You Need to Do

You have N blocks available. You can build your column in **two ways**:

2.1 Option 1: Use a Single Block

- Pick one block and orient it however you want
- The stability = the smallest of its three dimensions

Example:

- Block with dimensions 10, 8, 3
- No matter how you rotate it, the smallest dimension is 3
- So this column has stability = **3**

2.2 Option 2: Stack Two Blocks

- You can only stack two blocks if they have the **same base dimensions**
- When stacked, their heights add up
- The stability = minimum of (base dimensions, combined height)

Example:

- Block A: $10 \times 8 \times 3$ (base: 10×8 , height: 3)
- Block B: $10 \times 8 \times 5$ (base: 10×8 , height: 5)
- These can stack! Combined height: $3 + 5 = 8$
- Stability = $\min(10, 8) = \mathbf{8}$

Much better than using them separately!

3 Technical Details

3.1 How Blocks Work

Each block has three dimensions. We always arrange them as (x, y, z) where:

- $x \geq y \geq z$ (largest to smallest)
- Think of x and y as the base, z as the height

3.2 Stacking Rules

Two blocks can stack **only if**:

Block $i : (x_1, y_1, z_1)$

Block $j : (x_2, y_2, z_2)$

Can stack if: $x_1 = x_2$ AND $y_1 = y_2$

When stacked, the usable diameter is:

$$\text{diameter} = \min(y, z_1 + z_2)$$

(We use y because $x \geq y$ always, so y is the limiting base dimension)

4 Required Implementation

CRITICAL REQUIREMENT

You **MUST** use a hash table (dictionary) to group blocks that have the same base dimensions (x, y) . This allows you to quickly find which blocks can be stacked together.

Why? Without a hash table, you'd need to compare every block with every other block $O(N^2)$. With a hash table, you can do it in $O(N)$ time!

5 Input Format

Read from `input.txt`:

- **Line 1:** Number of blocks N where $1 \leq N \leq 10^5$
- **Next N lines:** Three space-separated integers representing block dimensions where $1 \leq \text{dimension} \leq 10^9$

Example:

```
4
10 8 3
8 10 5
6 6 4
6 6 3
```

6 Output Format

Write to `output.txt` (exactly 3 lines):

1. **Line 1:** Number of blocks used (1 or 2)
2. **Line 2:** Block index/indices (1-based, space-separated if two blocks, in ascending order)
3. **Line 3:** Maximum usable diameter achieved

Example:

```
2
1 2
8
```

If multiple solutions have the same maximum diameter, any valid one is acceptable.

7 Submission Guidelines

1. Create a new **public** repository named `ds-hw4-column-optimization`
2. Your repository should contain:

```
ds-hw4-column-optimization/
  README.md           (brief explanation of your approach)
  solution.py         (or solution.cpp, solution.java, etc.)
  input.txt           (sample test input)
  output.txt          (corresponding output)
```

3. Your code should:
 - Read from `input.txt` in the same directory
 - Write to `output.txt` in the same directory
 - Run without errors on standard environments
4. **Submit the GitHub repository link** via the SES.

Good luck!